

UNIVERSITÉ DU QUÉBEC À CHICOUTIMI

DÉPARTEMENT D'INFORMATIQUE ET DE MATHÉMATIQUE

8CLD876 - CONCEPTION/ARCHITECTURE DES SYSTÈMES INFONUAGIQUE

Protocole expérimental commun : Spring MVC, Spring WebFlux et Virtual Threads

Étudiant

BONNEAU Mathieu

GALICE Mahaut

JOURDAIN Gregory

Remis à

Hamid MCHEICK

28 Mai 2026

UQAC

Université du Québec
à Chicoutimi

Table des matières

1	Contexte	2
2	Problème étudié	2
3	Choix de l'environnement commun	2
4	Architecture technique commune	3
5	Application commune	3
6	Contrat API commun	4
7	Versions comparées	4
8	Configuration des ressources	5
9	Scénarios de test	5
10	Plan de charge	6
11	Métriques collectées	6
12	Outils utilisés	7
13	Implémentation pratique	7
14	Format des résultats	8
15	Références principales	8

1 Contexte

Le projet compare trois approches de développement backend dans l'écosystème Spring, à partir des principes introduits par l'article *SEDA : An Architecture for Well-Conditioned, Scalable Internet Services*. L'objectif est d'étudier comment des frameworks web modernes permettent de construire des services cloud capables de rester stables sous forte charge.

Élément	Description
Sujet général	Web Frameworks : Spring MVC, Spring WebFlux, Spring Data JPA, Virtual Threads
Article de départ	SEDA : architecture événementielle par <i>stages</i> , files explicites et contrôle de surcharge
Objectif du projet	Comparer plusieurs modèles d'exécution pour applications web cloud
Principe de comparaison	Même API, mêmes données, mêmes tests, mêmes métriques ; seul le modèle d'exécution change

2 Problème étudié

La question principale est la suivante : **comment comparer équitablement plusieurs frameworks web modernes afin d'évaluer leur scalabilité, leur robustesse et leur comportement sous surcharge ?**

Risque	Comparer des applications trop différentes et attribuer à tort les résultats au framework.
Solution	Définir une base expérimentale commune : mêmes endpoints, même base, mêmes données, mêmes scénarios et mêmes limites de ressources.
Ce que l'on isole	L'effet du modèle d'exécution : bloquant classique, réactif non bloquant ou threads légers.
Critères observés	Débit, latence, surcharge, CPU, mémoire, threads, erreurs et complexité de développement.

3 Choix de l'environnement commun

Les tests seront exécutés sur une machine virtuelle commune afin de garantir la reproductibilité. Toutes les versions seront lancées dans les mêmes conditions.

Paramètre	Choix retenu
Système	Ubuntu Server 24.04 LTS
CPU	4 vCPU
Mémoire	8 Go RAM
Stockage	40 Go
Java	Java 21
Build	Maven
Conteneurisation	Docker et Docker Compose
Tests de charge	k6
Outils système	Git, curl, jq, Docker Stats, outils JDK
Logs	Niveau WARN pendant les benchmarks

4 Architecture technique commune

Chaque application sera testée séparément. La base PostgreSQL et le service externe simulé seront communs.

Composant	Port	Rôle
Spring MVC + JPA	8081	Version classique, impérative et bloquante
Spring WebFlux + R2DBC	8082	Version réactive et non bloquante
Spring MVC + Virtual Threads	8083	Version impérative avec threads légers Java 21
PostgreSQL	5432	Base de données commune
Service externe simulé	9090	Simulation d'appels réseau avec latence contrôlée

Organisation proposée du dépôt :

```
project-cloud-web-frameworks/
  mvc-jpa/
  webflux-r2dbc/
  mvc-virtual-threads/
  common-db/
  mock-service/
  benchmark/
```

5 Application commune

L'application choisie est une API de gestion de commandes. Elle représente un backend d'entreprise classique avec lectures, écritures, accès base de données et appels à un service externe.

Entité	Champs principaux
customers	id, name, email
products	id, name, price, stock
orders	id, customer_id, status, created_at
order_items	id, order_id, product_id, quantity

Donnée	Volume	Objectif
Clients	10 000	Simuler une base utilisateur réaliste
Produits	5 000	Fournir un catalogue de taille moyenne
Commandes	100 000	Tester les lectures et écritures métier
Lignes de commande	300 000 à 500 000	Tester les relations et lectures composées

6 Contrat API commun

Les trois versions devront exposer exactement les mêmes endpoints, avec les mêmes méthodes HTTP, formats JSON et codes de retour.

Endpoint	Type	Objectif
GET /products	Lecture	Liste de produits
GET /products/{id}	Lecture	Consultation d'un produit
GET /orders/{id}	Lecture composée	Commande avec lignes associées
GET /customers/{id}/orders	Lecture composée	Commandes d'un client
POST /orders	Écriture	Création d'une commande
GET /external/recommendations/{id}	I/O-bound	Appel au service externe simulé
GET /stress/cpu?n=...	CPU-bound	Calcul local intensif

7 Versions comparées

Contraintes spécifiques pour WebFlux :

Version	Technologies	Règles d'implémentation
Spring MVC + JPA	Spring MVC, Spring Data JPA, Hibernate, JDBC, HikariCP	Code impératif; contrôleurs classiques; appels base et HTTP bloquants.
Spring WebFlux + R2DBC	Spring WebFlux, Project Reactor, Spring Data R2DBC, WebClient	Code réactif; retours en <code>Mono<T></code> ou <code>Flux<T></code> ; aucun appel bloquant.
Spring MVC + Virtual Threads	Spring MVC, Spring Data JPA, Java 21 Virtual Threads	Code impératif; activation de <code>spring.threads.virtual.enabled=true</code> .

Interdit	Raison
<code>Thread.sleep()</code>	Bloque un thread et fausse le modèle réactif
<code>.block()</code>	Transforme le flux réactif en appel bloquant
Spring Data JPA	JPA repose sur JDBC, donc sur des appels bloquants
Client HTTP bloquant	L'appel externe doit rester non bloquant via <code>WebClient</code>

8 Configuration des ressources

Les limites de ressources seront identiques pour chaque test afin d'éviter qu'une version bénéficie d'un avantage matériel.

Composant	CPU	RAM	Remarque
Application testée	2 vCPU	1,5 Go	Une seule application lancée par test
PostgreSQL	1 vCPU	1,5 Go	Base commune
Service externe simulé	0,5 vCPU	512 Mo	Latence contrôlée

La limite de connexions à la base sera fixée à 20 connexions pour les versions JPA et à une limite équivalente pour R2DBC. Ce choix évite qu'une version obtienne de meilleurs résultats uniquement grâce à un nombre plus élevé de connexions.

9 Scénarios de test

Scénario	Composition	Objectif
CRUD classique	60 % produits, 20 % commandes, 10 % commandes client, 10 % créations	Représenter une application métier standard
I/O-bound	Appels à <code>/external/recommendations/{id}</code> avec 100, 300 ou 500 ms de latence	Tester les attentes réseau et l'impact de WebFlux
CPU-bound	Appels à <code>/stress/cpu?n=...</code>	Tester le cas où le processeur devient le facteur limitant

10 Plan de charge

Niveau	Utilisateurs virtuels	Objectif
Faible	50	Vérifier le fonctionnement nominal
Moyen	200	Observer les premières différences
Fort	500	Étudier la montée en charge
Très fort	1000	Mesurer la robustesse
Surcharge	2000	Observer la dégradation du service

Chaque test comprendra 60 secondes de *warm-up*, puis 3 minutes de mesure. Chaque expérience sera répétée trois fois.

11 Métriques collectées

Catégorie	Métrique	Utilité
Performance	Requêtes par seconde	Mesurer le débit
Latence	Moyenne, médiane, P95, P99	Observer la latence normale et extrême
Fiabilité	Taux d'erreur HTTP	Détecter les échecs sous charge
Ressources	CPU et mémoire	Mesurer le coût d'exécution
Concurrence	Nombre de threads	Comparer les modèles d'exécution
Base de données	Connexions actives	Identifier une saturation PostgreSQL
Programmabilité	Classes, lignes de code, lisibilité, debug	Comparer la maintenabilité

12 Outils utilisés

Outil	Utilisation
Docker Compose	Lancement reproductible des applications, de PostgreSQL et du service externe
PostgreSQL	Base de données commune
k6	Tests de charge et export des métriques de latence, débit et erreurs
Spring Actuator	Exposition des métriques applicatives
Micrometer	Collecte standardisée des métriques Spring
Docker Stats	Observation CPU et mémoire
VisualVM / JDK Mission Control / jcmd	Analyse du nombre de threads et diagnostic JVM

Configuration minimale commune pour Actuator :

```
management.endpoints.web.exposure.include=health,metrics,prometheus
management.endpoint.health.show-details=always
```

13 Implémentation pratique

Exemple de commande :

```
k6 run -e BASE_URL=http://localhost:8082 benchmark/scenario-io.js
```

La variable `BASE_URL` sera modifiée selon la version testée. Les scripts de benchmark resteront identiques.

Étape	Action
1	Démarrer PostgreSQL et le service externe simulé avec Docker Compose
2	Démarrer une seule application à tester
3	Exécuter le script k6 avec l'URL cible
4	Sauvegarder les résultats en JSON ou CSV
5	Arrêter l'application testée
6	Répéter la procédure pour les deux autres versions

14 Format des résultats

Version	Scénario	VU	Req/s	Moy.	P95	P99	Erreurs
MVC JPA	CRUD	50					
WebFlux	CRUD	50					
Virtual Threads	CRUD	50					

Un second tableau pourra regrouper les métriques système :

Version	Scénario	CPU	RAM	Threads	Connexions DB
MVC JPA	CRUD				
WebFlux	CRUD				
Virtual Threads	CRUD				

L'objectif final ne sera pas simplement de désigner un framework meilleur que les autres, mais d'identifier les contextes dans lesquels chaque approche est pertinente. Spring MVC servira de référence classique, Spring WebFlux permettra d'étudier le modèle réactif non bloquant, et les Virtual Threads permettront d'évaluer une alternative moderne conservant la simplicité du style impératif.

15 Références principales

- [1] Welsh, Culler et Brewer, *SEDA : An Architecture for Well-Conditioned, Scalable Internet Services*, SOSP, 2001.
- [2] Documentation officielle Spring Framework : Spring MVC, Spring WebFlux et Project Reactor.
- [3] Documentation Spring Data R2DBC, pour les accès base de données non bloquants.
- [4] OpenJDK, *JEP 444 : Virtual Threads*, Java 21.
- [5] Documentation officielle de k6, pour la définition des tests de charge.
- [6] Documentation officielle Spring Boot Actuator et Micrometer, pour l'exposition des métriques applicatives. ““